

Layovered — Referral Backend: Step-by-Step Implementation Guide

Who this is for: the backend developer building the referral feature, written so you can follow it even if you've never built a referral system before. Everything is spelled out — the exact tables, columns, types, the SQL to create them, the API request/response shapes, and the logic for granting rewards.

0. The whole feature in one paragraph (read this first)

A user shares a referral link. A friend taps it, installs the app, signs in with Google/Apple, and runs their **first routing search**. At that exact moment we give the **referrer +5 searches** and the **friend (referee) +3 searches**. These bonus searches are *extra* on top of the 2 free searches everyone gets per day, and they **do not expire daily**. We must make sure nobody cheats (e.g. referring themselves, or using the same phone twice). That's the entire feature.

Three words you'll see a lot:

- **Referrer** = the existing user who shares the link.
 - **Referee** = the new friend who taps the link and installs.
 - **Activation** = the moment the referee runs their first routing search. Rewards are given *only* at activation — not when they install, not when they sign up.
-

1. The database — tables and fields

You will touch **4 tables**: extend 1 existing (**users**) and create 3 new ones (**referrals**, **search_bonus_ledger**, **referral_config**).

Here's how they relate, in plain words:

- **users** — your existing accounts. We add a few columns for referral.
- **referrals** — one row per "friend was invited" relationship. Tracks its status.
- **search_bonus_ledger** — a history log of every change to a user's bonus-search balance (so we can audit/debug).
- **referral_config** — one row holding the reward numbers, so you can change "+5" later without touching code.

1.1 Extend the existing **users** table

Add these 5 columns. Run as a migration (**ALTER TABLE**).

```
ALTER TABLE users ADD COLUMN referral_code    VARCHAR(12) UNIQUE;
ALTER TABLE users ADD COLUMN bonus_search_balance INTEGER NOT NULL DEFAULT
0;
ALTER TABLE users ADD COLUMN auth_provider    VARCHAR(20);    --
'google' | 'apple' | 'anonymous'
```

```
ALTER TABLE users ADD COLUMN primary_device_id VARCHAR(128);
ALTER TABLE users ADD COLUMN referred_by_user_id BIGINT REFERENCES
users(id);
```

Column	Type	Default	What it's for
referral_code	VARCHAR(12), UNIQUE	(set on signup)	The short code in this user's invite link, e.g. a7k9m2qx . Must be unique across all users.
bonus_search_balance	INTEGER, NOT NULL	0	How many bonus searches this user currently has. Persistent — never reset by the daily job . Goes up when they earn referrals, down when they use a bonus search.
auth_provider	VARCHAR(20)	NULL	How the user signed in: google , apple , or anonymous . We only reward google/apple users.
primary_device_id	VARCHAR(128)	NULL	The device this account belongs to. Used to block "same phone, new email" cheating.
referred_by_user_id	BIGINT, FK→users.id	NULL	If this user was referred, who referred them. (The real source of truth is the referrals table; this is just a convenient shortcut.)

[MySQL] Use **INT** instead of **INTEGER** (both work), and **BIGINT** for the FK to match your **users.id** type. **VARCHAR(12) UNIQUE** is the same.

Note on auth_provider: if you already store the login method somewhere else, you don't have to duplicate it — just make sure the activation code (Section 5) can read "is this user google/apple or anonymous?" Use whatever you already have.

1.2 New table: referrals

One row each time someone is invited. This is the heart of the feature.

```
CREATE TABLE referrals (
  id BIGSERIAL PRIMARY KEY,
  referrer_user_id BIGINT NOT NULL REFERENCES users(id),
  referee_user_id BIGINT REFERENCES users(id),           -- NULL until
the friend signs in
  referee_device_id VARCHAR(128),
  status VARCHAR(20) NOT NULL DEFAULT 'pending', -- 'pending' |
'rewarded' | 'rejected'
  reject_reason VARCHAR(40),                             -- why it was
rejected (NULL if not rejected)
  referrer_reward JSONB,                                   -- snapshot of
what the referrer got
  referee_reward JSONB,                                   -- snapshot of
```

```
what the referee got
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  rewarded_at     TIMESTAMPTZ,                -- when the
reward was given (NULL until rewarded)
  CONSTRAINT uq_referee_once UNIQUE (referee_user_id) -- a person can
be referred only ONCE
);

CREATE INDEX idx_referrals_referrer ON referrals (referrer_user_id);
```

Column	Type	What it's for
id	BIGSERIAL, PK	Auto-incrementing row id.
referrer_user_id	BIGINT, FK→users.id, NOT NULL	The user who shared the link.
referee_user_id	BIGINT, FK→users.id	The friend. NULL at install time if they haven't signed in yet; filled in once they do.
referee_device_id	VARCHAR(128)	The friend's device id, captured at install. Used for the same-device check.
status	VARCHAR(20), NOT NULL, default 'pending'	pending = invited but not yet activated; rewarded = reward given; rejected = blocked (cheating/invalid).
reject_reason	VARCHAR(40)	If rejected, why. One of: self_referral, same_device, not_authenticated, already_referred, cap_exceeded. NULL otherwise.
referrer_reward	JSONB	Snapshot of what the referrer received, e.g. {"type":"searches","value":5}. We store it so history is accurate even if the config changes later.
referee_reward	JSONB	Same, for the friend, e.g. {"type":"searches","value":3}.
created_at	TIMESTAMPTZ, NOT NULL	When the referral was first recorded (install time).
rewarded_at	TIMESTAMPTZ	When the reward was granted. NULL until it happens. Also used for the monthly cap.

Why UNIQUE (referee_user_id)? It guarantees one person can never be referred twice, even if two requests race. The database enforces it — you don't have to.

[MySQL] Replace BIGSERIAL PRIMARY KEY with BIGINT AUTO_INCREMENT PRIMARY KEY, JSONB with JSON, and TIMESTAMPTZ ... DEFAULT now() with TIMESTAMP ... DEFAULT CURRENT_TIMESTAMP.

1.3 New table: `search_bonus_ledger`

A history log. Every time a user's bonus balance changes — up (earned a reward) or down (spent a bonus search) — write one row here. This is your audit trail; if a balance ever looks wrong, this log tells you exactly how it got there.

```
CREATE TABLE search_bonus_ledger (  
  id          BIGSERIAL PRIMARY KEY,  
  user_id     BIGINT NOT NULL REFERENCES users(id),  
  delta       INTEGER NOT NULL,           -- the change: +5, +3, -1, etc.  
  reason      VARCHAR(30) NOT NULL,      -- 'referral_referrer' |  
  'referral_referee' | 'search_consumed' | 'manual_adjust'  
  referral_id BIGINT REFERENCES referrals(id), -- which referral caused  
  this (NULL for search_consumed)  
  balance_after INTEGER NOT NULL,        -- the user's  
  bonus_search_balance AFTER applying this delta  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE INDEX idx_ledger_user ON search_bonus_ledger (user_id);
```

Column	Type	What it's for
<code>id</code>	BIGSERIAL, PK	Row id.
<code>user_id</code>	BIGINT, FK→users.id	Whose balance changed.
<code>delta</code>	INTEGER	The change. <code>+5</code> when a referrer earns, <code>+3</code> when a referee earns, <code>-1</code> when a bonus search is used.
<code>reason</code>	VARCHAR(30)	Why it changed: <code>referral_referrer</code> , <code>referral_referee</code> , <code>search_consumed</code> , or <code>manual_adjust</code> (for manual fixes/clawbacks).
<code>referral_id</code>	BIGINT, FK→referrals.id	Links the change back to the referral that caused it. NULL when reason is <code>search_consumed</code> .
<code>balance_after</code>	INTEGER	The new balance right after this change. Lets you reconcile: the latest row's <code>balance_after</code> for a user must equal <code>users.bonus_search_balance</code> .
<code>created_at</code>	TIMESTAMPTZ	When it happened.

Rule you must follow: whenever you change `users.bonus_search_balance`, write a matching ledger row **in the same database transaction**. They must never drift apart.

1.4 New table: `referral_config`

A single row holding the reward settings. The point: marketing wants to change `"+5"` to `"+7"`, or later to `"₹500 coupon"` — you change this row, **not the code**.

```
CREATE TABLE referral_config (  
  id                INTEGER PRIMARY KEY DEFAULT 1,  
  referrer_reward   JSONB NOT NULL DEFAULT '{"type":"searches","value":5}',  
  referee_reward    JSONB NOT NULL DEFAULT '{"type":"searches","value":3}',  
  monthly_cap       INTEGER NOT NULL DEFAULT 10,  
  activation_event   VARCHAR(40) NOT NULL DEFAULT 'first_search',  
  CONSTRAINT only_one_row CHECK (id = 1)  
);  
  
-- create the single config row:  
INSERT INTO referral_config (id) VALUES (1);
```

Column	Meaning
referrer_reward	What the referrer gets, e.g. {"type":"searches","value":5}.
referee_reward	What the friend gets, e.g. {"type":"searches","value":3}.
monthly_cap	Max number of rewarded referrals one user can earn per calendar month (anti-abuse + cost control). Default 10.
activation_event	What triggers the reward. For now always first_search.

How the code uses it: read this row, look at `reward.type`. If `type == "searches"`, add `reward.value` to the user's `bonus_search_balance`. Write the code as a little dispatcher so that **later** you can add a `type == "coupon"` branch without changing anything else. (See Section 5.4.)

2. How searches work now (and after this change)

Today: every user gets **2 routing searches per day**. You already have (a) a daily limit and (b) an API that returns "searches remaining today". We are adding a **second bucket** of searches.

After this change, a user has two buckets:

- **Daily bucket** (`dailyRemaining`) — resets to 2 every day. (You already have this.)
- **Bonus bucket** (`bonus_search_balance`) — earned from referrals, **never resets**. (New.)

Total searches the user can do right now = `dailyRemaining` + `bonus_search_balance`.

Spending order (important): when a user runs a search, always spend from the **daily bucket first**. Only when the daily bucket is empty (0) do you spend from the bonus bucket. This way the "free daily" searches get used before the "earned" ones, which is what users expect.

Worked example:

- User has `dailyRemaining = 2`, `bonus = 8`. Total shown = 10.
- They search → daily becomes 1, bonus stays 8. (No ledger row — daily isn't tracked in the ledger.)
- They search again → daily becomes 0, bonus stays 8.
- They search a third time → daily is 0, so spend bonus → bonus becomes 7, **write a ledger row**
`delta=-1, reason=search_consumed, balance_after=7`.

- Tomorrow → daily resets to 2, bonus is still 7. Total = 9.

3. The API endpoints

You'll build/extend **3 endpoints** plus **1 hook** inside your existing search code.

All endpoints require the user to be logged in (use your existing auth). Below, "the user" means the authenticated caller.

3.1 GET /referral/me — the user's referral info

The app calls this to show the Invite screen (their link, how many searches they've earned, etc.).

Request: no body. Just the auth token.

Response 200:

```
{
  "referralCode": "a7k9m2qx",
  "referralLink": "https://layovered.chottu.link/get-app?referrerId=a7k9m2qx",
  "bonusBalance": 8,
  "rewardedThisMonth": 3,
  "monthlyCap": 10,
  "rewardConfig": {
    "referrerReward": { "type": "searches", "value": 5 },
    "refereeReward": { "type": "searches", "value": 3 }
  }
}
```

How to build each field:

- `referralCode` → `users.referral_code` for the caller.
- `referralLink` → the ChottuLink base + `?referrerId=` + their code. (Ajitesh's base is `https://layovered.chottu.link/get-app`.)
- `bonusBalance` → `users.bonus_search_balance`.
- `rewardedThisMonth` → `SELECT count(*) FROM referrals WHERE referrer_user_id = :me AND status = 'rewarded' AND rewarded_at >= date_trunc('month', now())`.
- `monthlyCap`, `rewardConfig` → from `referral_config`.

Response 401: if not logged in.

The app uses `rewardConfig` so the copy reads "+5 / +3" from the server. The app must **not** hardcode 5 and 3 — it shows whatever the config says. (That's how a future change to ₹500 just works.)

3.2 POST /referral/attribute — record the referral at install

When a friend installs via a referral link, the app reads `referrerId` from the deferred deep link (ChottuLink) and calls this to record "this install came from referrer X".

Request body:

```
{
  "referrerCode": "a7k9m2qx",
  "deviceId": "device-abc-123",
  "refereeUserId": 5567 // optional - null/omitted if the friend
  hasn't signed in yet
}
```

What the endpoint does (step by step):

1. Look up the referrer: `SELECT id FROM users WHERE referral_code = :referrerCode`. If none → respond 200 but do nothing (bad/old code; don't crash).
2. **Idempotency check:** if a referral row already exists for this `referee_device_id` (or `refereeUserId`), return it instead of creating a duplicate. (The app might call this more than once.)
3. Run the cheap guards and decide the row's status:
 - If `refereeUserId` is given and already exists in another referral's `referee_user_id` → create row with `status='rejected'`, `reject_reason='already_referred'`. (Or just skip — your call; recording it is better for debugging.)
 - If the friend's `deviceId` equals the referrer's `primary_device_id`, or `refereeUserId == referrer's id` → `status='rejected'`, `reject_reason='same_device' / 'self_referral'`.
 - Otherwise → `status='pending'`.
4. Insert the `referrals` row with `referrer_user_id`, `referee_user_id` (may be null), `referee_device_id`, `status`, `created_at = now()`.

Important: these guards are "best effort / cheap" here. The **real** anti-abuse decision happens later, at activation (Section 5), because that's when we know for sure the friend authenticated.

Response 200:

```
{ "status": "pending", "referralId": 8901 }
```

3.3 GET /searches/remaining — extend your existing endpoint

You already have an endpoint that returns searches remaining today. Change it to return **three** numbers so the app can show the counter chip and the "out of searches" wall.

Response 200:

```
{
  "dailyRemaining": 1,
  "bonusBalance": 8,
```

```
"totalRemaining": 9
}
```

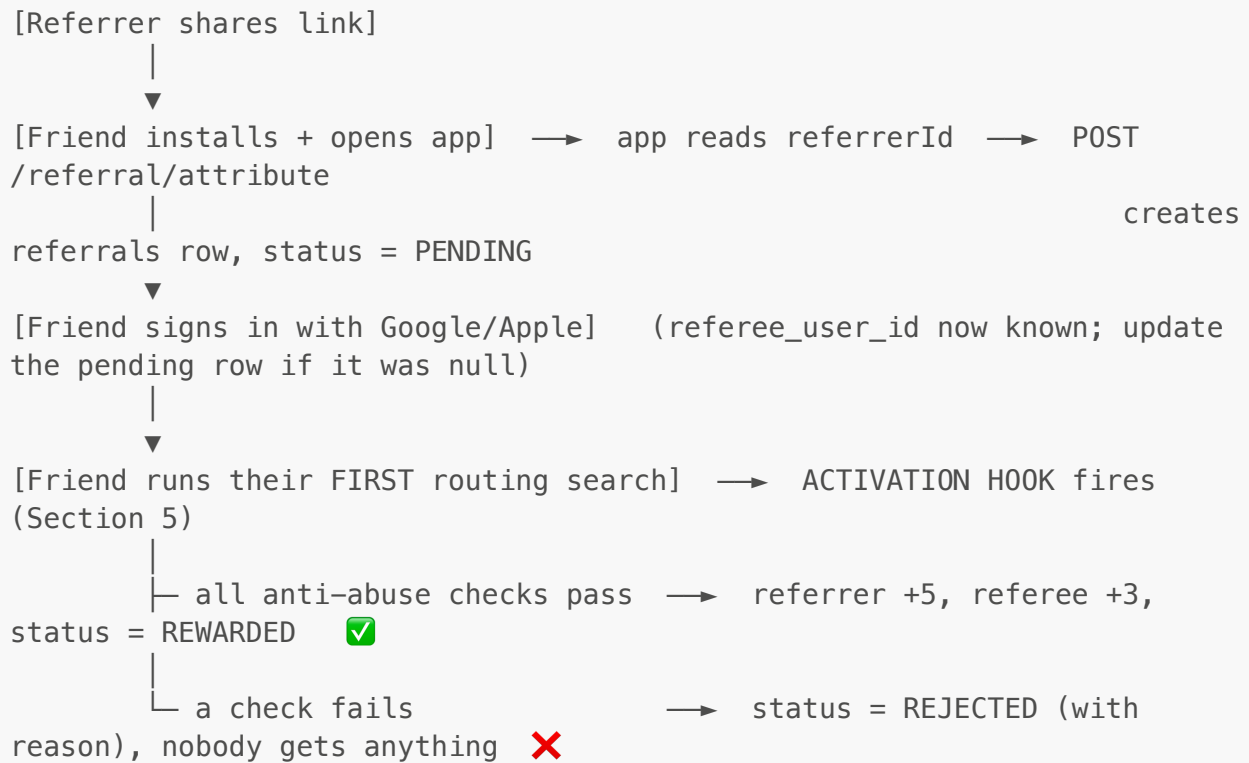
- `dailyRemaining` = your existing daily limit (2) minus what they've used today.
- `bonusBalance` = `users.bonus_search_balance`.
- `totalRemaining` = `dailyRemaining` + `bonusBalance`.

When `totalRemaining == 0`, the app shows the "out of searches" screen.

3.4 The activation hook — inside your existing "run a routing search" code

This isn't a new endpoint. It's logic you add **inside** the code that already runs when a user does a routing search. Full details in Section 5 — this is the most important part.

4. The referral lifecycle (the journey of one referral)



Key rule: **the reward is given at the friend's first search, not before.** Installing alone, or signing up alone, gives nothing. This is deliberate — it stops people farming fake installs.

5. The activation hook — step by step (the core logic)

This runs inside your existing search flow, **after** a routing search has successfully been performed by a user. Here it is as numbered steps, then as pseudocode.

5.1 The steps in plain English

1. The friend just ran a routing search. Ask: **is this their first ever routing search?** (If you don't already track this, the simplest signal is: do they have no prior search history, or a `first_search_done` flag that's still false.) If it's **not** their first search → do nothing, stop.
2. Find their pending referral: `SELECT * FROM referrals WHERE referee_user_id = :thisUser AND status = 'pending'`. If there's none → do nothing, stop. (This user wasn't referred, or was already processed.)
3. Run all the **anti-abuse checks** (Section 5.3). If **any** fails → set the referral `status='rejected'` with the matching `reject_reason`, grant nothing, stop.
4. If all checks pass → **grant the rewards inside ONE transaction** (Section 5.2).

5.2 Granting the reward — must be ONE transaction

"One transaction" means: either *all* of these succeed together, or *none* of them do. If the server crashes halfway, the database rolls back so you never half-credit someone. In code, wrap everything between `BEGIN` and `COMMIT` (or use your ORM's `transaction()` wrapper).

Inside the transaction, do exactly this:

```
BEGIN

-- read the reward amounts from config (Section 5.4)
referrerValue = config.referrer_reward.value    -- e.g. 5
refereeValue  = config.referee_reward.value     -- e.g. 3

-- 1) credit the referrer
UPDATE users SET bonus_search_balance = bonus_search_balance +
referrerValue
  WHERE id = referrer_user_id;
INSERT INTO search_bonus_ledger (user_id, delta, reason, referral_id,
balance_after, created_at)
  VALUES (referrer_user_id, referrerValue, 'referral_referrer',
:referralId,
          <referrer's new balance>, now());

-- 2) credit the referee (the friend)
UPDATE users SET bonus_search_balance = bonus_search_balance +
refereeValue
  WHERE id = referee_user_id;
INSERT INTO search_bonus_ledger (user_id, delta, reason, referral_id,
balance_after, created_at)
  VALUES (referee_user_id, refereeValue, 'referral_referee',
:referralId,
          <referee's new balance>, now());

-- 3) mark the referral done, with a snapshot of what was granted
UPDATE referrals
  SET status = 'rewarded',
      rewarded_at = now(),
      referrer_reward = '{"type":"searches","value":5}',
      referee_reward  = '{"type":"searches","value":3}'
  WHERE id = :referralId;
```

```
-- 4) (optional) record who referred the friend
UPDATE users SET referred_by_user_id = referrer_user_id WHERE id =
referee_user_id;

COMMIT
```

To get **<referrer's new balance>** for the ledger, either read the row back after the UPDATE, or use **UPDATE ... RETURNING bonus_search_balance** (Postgres) in the same statement.

[MySQL] **UPDATE ... RETURNING** isn't supported on older MySQL — just **SELECT bonus_search_balance** again inside the transaction after the UPDATE.

5.3 The anti-abuse checks (all must pass to grant)

Check these in order. The first one that fails sets the reject reason and stops the grant.

#	Check	If it fails, reject_reason
1	The friend signed in with Google or Apple — referee.auth_provider IN ('google','apple') . If they're still anonymous/skip-login → leave the referral pending (don't reject), so it can still reward later when they sign in.	not_authenticated (or keep pending)
2	The friend's account is genuinely new (created recently / within a referral window you decide, e.g. last 30 days) and isn't already referred. The UNIQUE(referee_user_id) constraint backs this up.	already_referred
3	The friend's device isn't already tied to another existing account — especially not the referrer's. Compare referee_device_id against users.primary_device_id of all users. This kills the "same phone, new email" trick.	same_device
4	Not a self-referral: referrer_user_id != referee_user_id .	self_referral
5	The referrer is under the monthly cap: count their rewarded referrals this month; must be < monthly_cap .	cap_exceeded

Cap special case: if the referrer is over their monthly cap, you may still give the **friend their +3** (a nice welcome) and just skip the referrer's +5. Record this in the snapshot so it's clear what happened. (Optional — confirm with Ajitesh; simplest version is to skip both.)

5.4 Reading the reward from config (so "+5" is changeable)

Don't write **+ 5** anywhere in code. Instead:

```
config = SELECT * FROM referral_config WHERE id = 1
reward = config.referrer_reward //
{"type":"searches","value":5}
```

```

switch (reward.type) {
  case "searches":
    user.bonus_search_balance += reward.value
    write ledger row
    break
  // case "coupon":      <-- add later, no other code changes needed
  //   issueCoupon(user, reward.value)
  // default:
  //   log error "unknown reward type" and DO NOT silently grant 0
}

```

This little **switch** is the whole reason marketing can later change the reward to a coupon without calling you back. Ship only the **searches** branch now; leave the others as comments.

5.5 Why it can't double-pay

Two safety nets:

1. After the first successful grant, the referral's **status** becomes **rewarded**. Step 2 of the hook only looks for **status = 'pending'** rows, so it can never grant twice for the same referral.
2. If two of the friend's searches somehow race at the same millisecond, the transaction + the **status** check + **UNIQUE(referee_user_id)** mean only one of them can flip pending→rewarded; the other finds no pending row. Net result: credited exactly once.

6. Edge cases — what to do when

Situation	What should happen
Friend installs but never signs in	Referral stays pending . No reward. (Optional: a cleanup job can expire pending rows older than, say, 30 days.)
Friend signs in days later, then searches	The hook fires on that first search and rewards normally.
Friend skips login (anonymous) and searches	No reward yet — referral stays pending until they actually sign in with Google/Apple.
You discover fraud <i>after</i> paying out	Don't delete rows. Add a manual_adjust ledger row with a negative delta to claw back the bonus, and set the referral status='rejected' .
Marketing changes +5 to +7, or to a ₹500 coupon	Update referral_config only. Schema and flow are untouched. New referrals use the new value; old referrals rows keep their snapshot.
Same person, two phones	Allowed (different devices). The same-device check only blocks the <i>same</i> device.

7. How to test it (do these before saying "done")

Run through each of these manually or as automated tests:

1. **Happy path.** User A shares link → User B installs (attribute called, row is `pending`) → B signs in with Google → B runs first search. ✅ Expect: A's `bonus_search_balance` +5, B's +3, referral `status='rewarded'`, two ledger rows written, `rewarded_at` set.
 2. **No double-pay.** B runs a *second* search. ✅ Expect: no change to balances, no new ledger rows.
 3. **Anonymous referee.** B skips login and searches. ✅ Expect: nobody credited, referral still `pending`. Then B signs in and searches → now credited.
 4. **Self-referral.** A installs via their own link. ✅ Expect: referral `rejected`, reason `self_referral`, no credit.
 5. **Same device.** B uses A's phone (same `deviceId`) with a new email. ✅ Expect: `rejected`, reason `same_device`.
 6. **Already referred.** Try to attribute B to a second referrer. ✅ Expect: blocked by `UNIQUE(referee_user_id) / already_referred`.
 7. **Monthly cap.** Give A 10 rewarded referrals this month, then an 11th activates. ✅ Expect: A does **not** get +5 (cap), reason `cap_exceeded` (friend may still get +3 if you chose that option).
 8. **Spending order.** User with `daily=2`, `bonus=3`. Do 5 searches. ✅ Expect: first 2 spend daily (no ledger), next 3 spend bonus (3 ledger rows `delta=-1`), 6th search refused.
 9. **API shape.** `GET /searches/remaining` returns all three fields and `total = daily + bonus`. `GET /referral/me` returns the code, link, balance, and config.
 10. **Reconciliation.** For any user, the latest `search_bonus_ledger.balance_after` equals `users.bonus_search_balance`. They must always match.
-

8. Build order (what to do first)

1. **Migrations** — create the 4 tables / columns (Section 1). Insert the single `referral_config` row.
2. **Assign `referral_code`** on signup, and backfill existing users.
3. **Extend `GET /searches/remaining`** (Section 3.3) + the spending order (Section 2). *Do this early* — the app's search-counter UI needs it and it doesn't depend on the rest.
4. **`GET /referral/me`** (Section 3.1).
5. **`POST /referral/attribute`** (Section 3.2).
6. **The activation hook + grant** (Section 5) — the core. Test hard (Section 7).
7. **Analytics events** — fire `invite_sent`, `referral_install`, `referral_activated`, `reward_granted` so we can measure the loop.

If you get stuck on any step, the matching Jira ticket in `REFERRAL_BACKEND_TICKETS.md` has the acceptance criteria, and `REFERRAL_LOOP_SPEC.md` explains the "why" behind every rule.